EXTEND WITH TOOLS

Connect to external tools with MCP

Configure MCP servers to extend your agent with external tools. Covers transport types, tool search for large tool sets, authentication, and error handling.

The **Model Context Protocol (MCP)** is an open standard for connecting AI agents to external tools and data sources. With MCP, your agent can query databases, integrate with APIs like Slack and GitHub, and connect to other services without writing custom tool implementations.

MCP servers can run as local processes, connect over HTTP, or execute directly within your SDK application.

> This page covers MCP configuration for the Agent SDK. To add MCP servers to the Claude Code CLI so they load in every project, see **MCP installation scopes**.

Quickstart

This example connects to the **Claude Code documentation** MCP server using **HTTP transport** and uses `allowedTools` with a wildcard to permit all tools from the server.

The agent connects to the documentation server, searches for information about hooks, and returns the results.

Add an MCP server

You can configure MCP servers in code when calling `query()`, or in a `.mcp.json` file loaded via `settingSources`.

In code

Pass MCP servers directly in the `mcpServers` option:

From a config file

Create a `.mcp.json` file at your project root. The file is picked up when the `project` setting source is enabled, which it is for default `query()` options. If you set `settingSources` explicitly, include `"project"` for this file to load:

```
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-filesystem",
"/Users/me/projects"]
    }
  }
}
```

Allow MCP tools

MCP tools require explicit permission before Claude can use them. Without permission, Claude will see that tools are available but won't be able to call them.

Tool naming convention

MCP tools follow the naming pattern `mcp__<server-name>__<tool-name>`. For example, a GitHub server named `"github"` with a `list_issues` tool becomes `mcp__github__list_issues`.

Auto-approve with allowedTools

Use `allowedTools` to auto-approve specific MCP tools so Claude can use them without a permission prompt:

```
const _ = {
  options: {
    mcpServers: {
      // your servers
    },
    allowedTools: [
      "mcp__github__*", // All tools from the github server
      "mcp__db__query", // Only the query tool from db server
      "mcp__slack__send_message" // Only send_message from slack
server
    ]
  }
};
```

Wildcards (`*`) let you allow all tools from a server without listing each one individually.

> **Prefer `allowedTools` over permission modes for MCP access.** `permissionMode: "acceptEdits"` does not auto-approve MCP tools (only file edits and filesystem Bash commands). `permissionMode: "bypassPermissions"` does auto-approve MCP tools but also disables other safety prompts unless an explicit `ask` **rule** matches, which is broader than necessary. A wildcard in `allowedTools` grants exactly the MCP server you want and nothing more. See **Permission modes** for a full comparison.

Discover available tools

To see what tools an MCP server provides, check the server's documentation or connect to the server and inspect the `system` init message:

Transport types

MCP servers communicate with your agent using different transport protocols. Check the server's documentation to see which transport it supports:

- If the docs give you a **command to run** (like `npx @modelcontextprotocol/server-github`), use stdio
- If the docs give you a **URL**, use HTTP or SSE
- If you're building your own tools in code, use an SDK MCP server

stdio servers

Local processes that communicate via stdin/stdout. Use this for MCP servers you run on the same machine:

In code

```
import { query } from "@anthropic-ai/claude-agent-sdk";

for await (const message of query({
  prompt: "Use the docs MCP server to explain what hooks are in Claude Code",
  options: {
    mcpServers: {
      "claude-code-docs": {
        type: "http",
        url: "https://code.claude.com/docs/mcp"
      }
    },
    allowedTools: ["mcp__claude-code-docs__*"]
  }
})) {
  if (message.type === "result" && message.subtype === "success") {
    console.log(message.result);
  }
}
```

.mcp.json

TypeScript

```
import { query } from "@anthropic-ai/claude-agent-sdk";

for await (const message of query({
  prompt: "List files in my project",
  options: {
    mcpServers: {
```

```
      filesystem: {
        command: "npx",
        args: ["-y", "@modelcontextprotocol/server-filesystem", "/Users/me/projects"]
      }
    },
    allowedTools: ["mcp__filesystem__*"]
  }
})) {
  if (message.type === "result" && message.subtype === "success") {
    console.log(message.result);
  }
}
```

Python

```
for await (const message of query({ prompt: "...", options })) {
  if (message.type === "system" && message.subtype === "init") {
    console.log("Available MCP tools:", message.mcp_servers);
  }
}
```

TypeScript Python

```
const _ = {
  options: {
    mcpServers: {
      github: {
        command: "npx",
        args: ["-y", "@modelcontextprotocol/server-github"],
        env: {
          GITHUB_TOKEN: process.env.GITHUB_TOKEN
        }
      }
    },
    allowedTools: ["mcp__github__list_issues", "mcp__github__search_issues"]
  }
};
```

```
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {
        "GITHUB_TOKEN": "${GITHUB_TOKEN}"
      }
    }
  }
}
```

TypeScript Python

```
const _ = {
  options: {
    mcpServers: {
      "remote-api": {
        type: "sse",
        url: "https://api.example.com/mcp/sse",
        headers: {
          Authorization: `Bearer ${process.env.API_TOKEN}`
        }
      }
    },
    allowedTools: ["mcp__remote-api__*"]
  }
};
```

```
{
  "mcpServers": {
    "remote-api": {
      "type": "sse",
      "url": "https://api.example.com/mcp/sse",
      "headers": {
        "Authorization": "Bearer ${API_TOKEN}"
      }
    }
  }
}
```

TypeScript Python

```
const _ = {
  options: {
    mcpServers: {
      github: {
        command: "npx",
        args: ["-y", "@modelcontextprotocol/server-github"],
        env: {
          GITHUB_TOKEN: process.env.GITHUB_TOKEN
        }
      }
    },
    allowedTools: ["mcp__github__list_issues"]
  }
};
```

```
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {
        "GITHUB_TOKEN": "${GITHUB_TOKEN}"
      }
    }
  }
}
```

The `${GITHUB_TOKEN}` syntax expands environment variables at runtime.

TypeScript | Python

```
const _ = {
  options: {
    mcpServers: {
      "secure-api": {
        type: "http",
        url: "https://api.example.com/mcp",
        headers: {
          Authorization: `Bearer ${process.env.API_TOKEN}`
        }
      }
    },
    allowedTools: ["mcp__secure-api__*"]
  }
};
```

```
{
  "mcpServers": {
    "secure-api": {
      "type": "http",
      "url": "https://api.example.com/mcp",
      "headers": {
        "Authorization": "Bearer ${API_TOKEN}"
      }
    }
  }
}
```

The `${API_TOKEN}` syntax expands environment variables at runtime.

```
// After completing OAuth flow in your app
const accessToken = await getAccessTokenFromOAuthFlow();

const options = {
  mcpServers: {
    "oauth-api": {
      type: "http",
      url: "https://api.example.com/mcp",
      headers: {
        Authorization: `Bearer ${accessToken}`
      }
    }
  },
  allowedTools: ["mcp__oauth-api__*"]
};
```

```typescript
import { query } from "@anthropic-ai/claude-agent-sdk";

for await (const message of query({
  prompt: "List the 3 most recent issues in anthropics/claude-code",
  options: {
    mcpServers: {
      github: {
        command: "npx",
        args: ["-y", "@modelcontextprotocol/server-github"],
        env: {
          GITHUB_TOKEN: process.env.GITHUB_TOKEN
        }
      }
    },
    allowedTools: ["mcp__github__list_issues"]
  }
})) {
  // Verify MCP server connected successfully
  if (message.type === "system" && message.subtype === "init") {
    console.log("MCP servers:", message.mcp_servers);
  }

  // Log when Claude calls an MCP tool
  if (message.type === "assistant") {
    for (const block of message.message.content) {
      if (block.type === "tool_use" && block.name.startsWith("mcp__")) {
        console.log("MCP tool called:", block.name);
      }
    }
  }

  // Print the final result
  if (message.type === "result" && message.subtype === "success") {
    console.log(message.result);
  }
}
```

```
import { query } from "@anthropic-ai/claude-agent-sdk";

// Connection string from environment variable
const connectionString = process.env.DATABASE_URL;

for await (const message of query({
  // Natural language query - Claude writes the SQL
  prompt: "How many users signed up last week? Break it down by day.",
  options: {
    mcpServers: {
      postgres: {
        command: "npx",
        // Pass connection string as argument to the server
        args: ["-y", "@modelcontextprotocol/server-postgres", connectionString]
      }
    },
    // Allow only read queries, not writes
    allowedTools: ["mcp__postgres__query"]
  }
})) {
  if (message.type === "result" && message.subtype === "success") {
    console.log(message.result);
  }
}
```

```
import { query } from "@anthropic-ai/claude-agent-sdk";

for await (const message of query({
  prompt: "Process data",
  options: {
    mcpServers: {
      "data-processor": dataServer
    }
  }
})) {
  if (message.type === "system" && message.subtype === "init") {
    const failedServers = message.mcp_servers.filter((s) => s.status !==
```

```
"connected");

    if (failedServers.length > 0) {
      console.warn("Failed to connect:", failedServers);
    }
  }

  if (message.type === "result" && message.subtype === "error_during_execution") {
    console.error("Execution failed");
  }
}
```

HTTP/SSE servers

Use HTTP or SSE for cloud-hosted MCP servers and remote APIs:

In code

```
import { query } from "@anthropic-ai/claude-agent-sdk";

for await (const message of query({
  prompt: "Use the docs MCP server to explain what hooks are in Claude Code",
  options: {
    mcpServers: {
      "claude-code-docs": {
        type: "http",
        url: "https://code.claude.com/docs/mcp"
      }
    },
    allowedTools: ["mcp__claude-code-docs__*"]
  }
})) {
  if (message.type === "result" && message.subtype === "success") {
    console.log(message.result);
  }
}
```

.mcp.json

TypeScript

```
import { query } from "@anthropic-ai/claude-agent-sdk";

for await (const message of query({
  prompt: "List files in my project",
  options: {
    mcpServers: {
      filesystem: {
        command: "npx",
        args: ["-y", "@modelcontextprotocol/server-filesystem", "/Users/me/projects"]
      }
    },
    allowedTools: ["mcp__filesystem__*"]
  }
})) {
  if (message.type === "result" && message.subtype === "success") {
```

```
    console.log(message.result);
  }
}
```

Python

```
for await (const message of query({ prompt: "...", options })) {
  if (message.type === "system" && message.subtype === "init") {
    console.log("Available MCP tools:", message.mcp_servers);
  }
}
```

```
const _ = {
  options: {
    mcpServers: {
      github: {
        command: "npx",
        args: ["-y", "@modelcontextprotocol/server-github"],
        env: {
          GITHUB_TOKEN: process.env.GITHUB_TOKEN
        }
      }
    },
    allowedTools: ["mcp__github__list_issues", "mcp__github__search_issues"]
  }
};
```

```
const _ = {
  options: {
    mcpServers: {
      "remote-api": {
        type: "sse",
        url: "https://api.example.com/mcp/sse",
        headers: {
          Authorization: `Bearer ${process.env.API_TOKEN}`
        }
      }
    },
    allowedTools: ["mcp__remote-api__*"]
  }
};
```

```
const _ = {
  options: {
    mcpServers: {
      github: {
        command: "npx",
        args: ["-y", "@modelcontextprotocol/server-github"],
        env: {
          GITHUB_TOKEN: process.env.GITHUB_TOKEN
        }
      }
    },
    allowedTools: ["mcp__github__list_issues"]
  }
};
```

```
const _ = {
  options: {
    mcpServers: {
      "secure-api": {
        type: "http",
        url: "https://api.example.com/mcp",
        headers: {
          Authorization: `Bearer ${process.env.API_TOKEN}`
        }
      }
    },
    allowedTools: ["mcp__secure-api__*"]
  }
};
```

```
const _ = {
  options: {
    mcpServers: {
      github: {
        command: "npx",
        args: ["-y", "@modelcontextprotocol/server-github"],
        env: {
          GITHUB_TOKEN: process.env.GITHUB_TOKEN
        }
      }
    },
    allowedTools: ["mcp__github__list_issues", "mcp__github__search_issues"]
  }
};
```

```
const _ = {
  options: {
    mcpServers: {
      github: {
        command: "npx",
        args: ["-y", "@modelcontextprotocol/server-github"],
        env: {
          GITHUB_TOKEN: process.env.GITHUB_TOKEN
        }
```

```
        }
      },
      allowedTools: ["mcp__github__list_issues", "mcp__github__search_issues"]
    }
  };
```

```
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {
        "GITHUB_TOKEN": "${GITHUB_TOKEN}"
      }
    }
  }
}
```

TypeScript Python

```
const _ = {
  options: {
    mcpServers: {
      "remote-api": {
        type: "sse",
```

```
        url: "https://api.example.com/mcp/sse",
        headers: {
          Authorization: `Bearer ${process.env.API_TOKEN}`
        }
      }
    },
    allowedTools: ["mcp__remote-api__*"]
  }
};
```

```
{
  "mcpServers": {
    "remote-api": {
      "type": "sse",
      "url": "https://api.example.com/mcp/sse",
      "headers": {
        "Authorization": "Bearer ${API_TOKEN}"
      }
    }
  }
}
```

TypeScript Python

```
const _ = {
```

```
    options: {
      mcpServers: {
        github: {
          command: "npx",
          args: ["-y", "@modelcontextprotocol/server-github"],
          env: {
            GITHUB_TOKEN: process.env.GITHUB_TOKEN
          }
        }
      },
      allowedTools: ["mcp__github__list_issues"]
    }
  };
```

```
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {
        "GITHUB_TOKEN": "${GITHUB_TOKEN}"
      }
    }
  }
}
```

The `${GITHUB_TOKEN}` syntax expands environment variables at runtime.

TypeScript Python

```
const _ = {
  options: {
    mcpServers: {
      "secure-api": {
        type: "http",
        url: "https://api.example.com/mcp",
        headers: {
          Authorization: `Bearer ${process.env.API_TOKEN}`
        }
      }
    },
    allowedTools: ["mcp__secure-api__*"]
  }
};
```

```
{
  "mcpServers": {
    "secure-api": {
      "type": "http",
      "url": "https://api.example.com/mcp",
      "headers": {
        "Authorization": "Bearer ${API_TOKEN}"
      }
    }
  }
}
```

The `${API_TOKEN}` syntax expands environment variables at runtime.

```
// After completing OAuth flow in your app
const accessToken = await getAccessTokenFromOAuthFlow();

const options = {
  mcpServers: {
    "oauth-api": {
      type: "http",
      url: "https://api.example.com/mcp",
      headers: {
        Authorization: `Bearer ${accessToken}`
      }
    }
  },
  allowedTools: ["mcp__oauth-api__*"]
};
```

```
import { query } from "@anthropic-ai/claude-agent-sdk";

for await (const message of query({
  prompt: "List the 3 most recent issues in anthropics/claude-code",
  options: {
    mcpServers: {
      github: {
        command: "npx",
        args: ["-y", "@modelcontextprotocol/server-github"],
        env: {
          GITHUB_TOKEN: process.env.GITHUB_TOKEN
        }
      }
    },
    allowedTools: ["mcp__github__list_issues"]
  }
})) {
  // Verify MCP server connected successfully
  if (message.type === "system" && message.subtype === "init") {
    console.log("MCP servers:", message.mcp_servers);
  }
```

```
  // Log when Claude calls an MCP tool
  if (message.type === "assistant") {
    for (const block of message.message.content) {
      if (block.type === "tool_use" && block.name.startsWith("mcp__")) {
        console.log("MCP tool called:", block.name);
      }
    }
  }

  // Print the final result
  if (message.type === "result" && message.subtype === "success") {
    console.log(message.result);
  }
}
```

```
import { query } from "@anthropic-ai/claude-agent-sdk";

// Connection string from environment variable
const connectionString = process.env.DATABASE_URL;

for await (const message of query({
  // Natural language query - Claude writes the SQL
  prompt: "How many users signed up last week? Break it down by day.",
  options: {
    mcpServers: {
      postgres: {
        command: "npx",
        // Pass connection string as argument to the server
        args: ["-y", "@modelcontextprotocol/server-postgres", connectionString]
      }
    },
    // Allow only read queries, not writes
    allowedTools: ["mcp__postgres__query"]
  }
})) {
  if (message.type === "result" && message.subtype === "success") {
    console.log(message.result);
  }
```

```
}
```

```
import { query } from "@anthropic-ai/claude-agent-sdk";

for await (const message of query({
  prompt: "Process data",
  options: {
    mcpServers: {
      "data-processor": dataServer
    }
  }
})) {
  if (message.type === "system" && message.subtype === "init") {
    const failedServers = message.mcp_servers.filter((s) => s.status !==
"connected");

    if (failedServers.length > 0) {
      console.warn("Failed to connect:", failedServers);
    }
  }

  if (message.type === "result" && message.subtype === "error_during_execution") {
    console.error("Execution failed");
  }
}
```

For the streamable HTTP transport, use `"type": "http"` instead. In `.mcp.json` and other JSON config files, `"streamable-http"` is accepted as an alias for `"http"`. The programmatic `mcpServers` option accepts only `"http"`.

SDK MCP servers

Define custom tools directly in your application code instead of running a separate server process. See the **custom tools guide** for implementation details.

MCP tool search

When you have many MCP tools configured, tool definitions can consume a significant portion of your context window. Tool search solves this by withholding tool definitions from context and loading only the ones Claude needs for each turn.

Tool search is enabled by default. See **Tool search** for configuration options and details.

For more detail, including best practices and using tool search with custom SDK tools, see the **tool search guide**.

Authentication

Most MCP servers require authentication to access external services. Pass credentials through environment variables in the server configuration.

Pass credentials via environment variables

Use the `env` field to pass API keys, tokens, and other credentials to the MCP server:

In code

```
import { query } from "@anthropic-ai/claude-agent-sdk";

for await (const message of query({
  prompt: "Use the docs MCP server to explain what hooks are in Claude Code",
  options: {
    mcpServers: {
      "claude-code-docs": {
        type: "http",
        url: "https://code.claude.com/docs/mcp"
      }
    },
    allowedTools: ["mcp__claude-code-docs__*"]
  }
```

```
})) {
  if (message.type === "result" && message.subtype === "success") {
    console.log(message.result);
  }
}
```

.mcp.json

TypeScript

```
import { query } from "@anthropic-ai/claude-agent-sdk";

for await (const message of query({
  prompt: "List files in my project",
  options: {
    mcpServers: {
      filesystem: {
        command: "npx",
        args: ["-y", "@modelcontextprotocol/server-filesystem", "/Users/me/projects"]
      }
    },
    allowedTools: ["mcp__filesystem__*"]
  }
})) {
  if (message.type === "result" && message.subtype === "success") {
    console.log(message.result);
  }
}
```

Python

```
for await (const message of query({ prompt: "...", options })) {
  if (message.type === "system" && message.subtype === "init") {
    console.log("Available MCP tools:", message.mcp_servers);
  }
}
```

```
const _ = {
  options: {
    mcpServers: {
      github: {
        command: "npx",
        args: ["-y", "@modelcontextprotocol/server-github"],
        env: {
          GITHUB_TOKEN: process.env.GITHUB_TOKEN
        }
      }
    },
    allowedTools: ["mcp__github__list_issues", "mcp__github__search_issues"]
  }
};
```

```
const _ = {
  options: {
    mcpServers: {
```

```
      "remote-api": {
        type: "sse",
        url: "https://api.example.com/mcp/sse",
        headers: {
          Authorization: `Bearer ${process.env.API_TOKEN}`
        }
      }
    },
    allowedTools: ["mcp__remote-api__*"]
  }
};
```

```
const _ = {
  options: {
    mcpServers: {
      github: {
        command: "npx",
        args: ["-y", "@modelcontextprotocol/server-github"],
        env: {
          GITHUB_TOKEN: process.env.GITHUB_TOKEN
        }
      }
    },
    allowedTools: ["mcp__github__list_issues"]
  }
};
```

```
const _ = {
  options: {
    mcpServers: {
      "secure-api": {
        type: "http",
        url: "https://api.example.com/mcp",
        headers: {
          Authorization: `Bearer ${process.env.API_TOKEN}`
        }
      }
    },
    allowedTools: ["mcp__secure-api__*"]
  }
};
```

```
const _ = {
  options: {
    mcpServers: {
      github: {
        command: "npx",
        args: ["-y", "@modelcontextprotocol/server-github"],
        env: {
          GITHUB_TOKEN: process.env.GITHUB_TOKEN
        }
      }
    },
    allowedTools: ["mcp__github__list_issues", "mcp__github__search_issues"]
  }
};
```

```
const _ = {
  options: {
    mcpServers: {
      github: {
        command: "npx",
        args: ["-y", "@modelcontextprotocol/server-github"],
        env: {
          GITHUB_TOKEN: process.env.GITHUB_TOKEN
        }
      }
    },
    allowedTools: ["mcp__github__list_issues", "mcp__github__search_issues"]
  }
};
```

```
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {
        "GITHUB_TOKEN": "${GITHUB_TOKEN}"
      }
    }
  }
}
```

```
  const _ = {
    options: {
      mcpServers: {
        github: {
          command: "npx",
          args: ["-y", "@modelcontextprotocol/server-github"],
          env: {
            GITHUB_TOKEN: process.env.GITHUB_TOKEN
          }
        }
      },
      allowedTools: ["mcp__github__list_issues", "mcp__github__search_issues"]
    }
  };
```

```
  const _ = {
    options: {
```

```
    mcpServers: {
      "remote-api": {
        type: "sse",
        url: "https://api.example.com/mcp/sse",
        headers: {
          Authorization: `Bearer ${process.env.API_TOKEN}`
        }
      }
    },
    allowedTools: ["mcp__remote-api__*"]
  }
};
```

```
const _ = {
  options: {
    mcpServers: {
      github: {
        command: "npx",
        args: ["-y", "@modelcontextprotocol/server-github"],
        env: {
          GITHUB_TOKEN: process.env.GITHUB_TOKEN
        }
      }
    },
    allowedTools: ["mcp__github__list_issues"]
  }
};
```

```
const _ = {
  options: {
    mcpServers: {
      "secure-api": {
        type: "http",
        url: "https://api.example.com/mcp",
        headers: {
          Authorization: `Bearer ${process.env.API_TOKEN}`
        }
      }
    },
    allowedTools: ["mcp__secure-api__*"]
  }
};
```

```
const _ = {
  options: {
    mcpServers: {
      "remote-api": {
        type: "sse",
        url: "https://api.example.com/mcp/sse",
        headers: {
          Authorization: `Bearer ${process.env.API_TOKEN}`
        }
      }
    },
    allowedTools: ["mcp__remote-api__*"]
  }
};
```

```
const _ = {
  options: {
    mcpServers: {
      "remote-api": {
        type: "sse",
        url: "https://api.example.com/mcp/sse",
        headers: {
          Authorization: `Bearer ${process.env.API_TOKEN}`
        }
      }
    },
    allowedTools: ["mcp__remote-api__*"]
  }
};
```

```
{
  "mcpServers": {
    "remote-api": {
      "type": "sse",
      "url": "https://api.example.com/mcp/sse",
      "headers": {
        "Authorization": "Bearer ${API_TOKEN}"
      }
    }
  }
}
```

TypeScript Python

```
const _ = {
  options: {
    mcpServers: {
      github: {
        command: "npx",
        args: ["-y", "@modelcontextprotocol/server-github"],
        env: {
          GITHUB_TOKEN: process.env.GITHUB_TOKEN
        }
      }
    },
    allowedTools: ["mcp__github__list_issues"]
  }
};
```

```
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {
        "GITHUB_TOKEN": "${GITHUB_TOKEN}"
      }
    }
  }
}
```

The `${GITHUB_TOKEN}` syntax expands environment variables at runtime.

TypeScript Python

```
const _ = {
  options: {
    mcpServers: {
      "secure-api": {
        type: "http",
        url: "https://api.example.com/mcp",
        headers: {
          Authorization: `Bearer ${process.env.API_TOKEN}`
        }
      }
    },
    allowedTools: ["mcp__secure-api__*"]
  }
};
```

```
{
  "mcpServers": {
    "secure-api": {
      "type": "http",
      "url": "https://api.example.com/mcp",
      "headers": {
        "Authorization": "Bearer ${API_TOKEN}"
      }
    }
  }
}
```

The `${API_TOKEN}` syntax expands environment variables at runtime.

```
// After completing OAuth flow in your app
const accessToken = await getAccessTokenFromOAuthFlow();

const options = {
  mcpServers: {
    "oauth-api": {
      type: "http",
      url: "https://api.example.com/mcp",
      headers: {
        Authorization: `Bearer ${accessToken}`
      }
    }
  },
  allowedTools: ["mcp__oauth-api__*"]
};
```

```typescript
import { query } from "@anthropic-ai/claude-agent-sdk";

for await (const message of query({
  prompt: "List the 3 most recent issues in anthropics/claude-code",
  options: {
    mcpServers: {
      github: {
        command: "npx",
        args: ["-y", "@modelcontextprotocol/server-github"],
        env: {
          GITHUB_TOKEN: process.env.GITHUB_TOKEN
        }
      }
    },
    allowedTools: ["mcp__github__list_issues"]
  }
})) {
  // Verify MCP server connected successfully
  if (message.type === "system" && message.subtype === "init") {
    console.log("MCP servers:", message.mcp_servers);
  }

  // Log when Claude calls an MCP tool
  if (message.type === "assistant") {
    for (const block of message.message.content) {
      if (block.type === "tool_use" && block.name.startsWith("mcp__")) {
        console.log("MCP tool called:", block.name);
      }
    }
  }

  // Print the final result
  if (message.type === "result" && message.subtype === "success") {
    console.log(message.result);
  }
}
```

```
import { query } from "@anthropic-ai/claude-agent-sdk";

// Connection string from environment variable
const connectionString = process.env.DATABASE_URL;

for await (const message of query({
  // Natural language query - Claude writes the SQL
  prompt: "How many users signed up last week? Break it down by day.",
  options: {
    mcpServers: {
      postgres: {
        command: "npx",
        // Pass connection string as argument to the server
        args: ["-y", "@modelcontextprotocol/server-postgres", connectionString]
      }
    },
    // Allow only read queries, not writes
    allowedTools: ["mcp__postgres__query"]
  }
})) {
  if (message.type === "result" && message.subtype === "success") {
    console.log(message.result);
  }
}
```

```
import { query } from "@anthropic-ai/claude-agent-sdk";

for await (const message of query({
  prompt: "Process data",
  options: {
    mcpServers: {
      "data-processor": dataServer
    }
  }
```

```
})) {
  if (message.type === "system" && message.subtype === "init") {
    const failedServers = message.mcp_servers.filter((s) => s.status !==
"connected");

    if (failedServers.length > 0) {
      console.warn("Failed to connect:", failedServers);
    }
  }

  if (message.type === "result" && message.subtype === "error_during_execution") {
    console.error("Execution failed");
  }
}
```

See **List issues from a repository** for a complete working example with debug logging.

HTTP headers for remote servers

For HTTP and SSE servers, pass authentication headers directly in the server configuration:

In code

```
import { query } from "@anthropic-ai/claude-agent-sdk";

for await (const message of query({
  prompt: "Use the docs MCP server to explain what hooks are in Claude Code",
  options: {
    mcpServers: {
      "claude-code-docs": {
        type: "http",
        url: "https://code.claude.com/docs/mcp"
      }
    },
    allowedTools: ["mcp__claude-code-docs__*"]
  }
})) {
  if (message.type === "result" && message.subtype === "success") {
```

```
    console.log(message.result);
  }
}
```

.mcp.json

TypeScript

```
import { query } from "@anthropic-ai/claude-agent-sdk";

for await (const message of query({
  prompt: "List files in my project",
  options: {
    mcpServers: {
      filesystem: {
        command: "npx",
        args: ["-y", "@modelcontextprotocol/server-filesystem", "/Users/me/projects"]
      }
    },
    allowedTools: ["mcp__filesystem__*"]
  }
})) {
  if (message.type === "result" && message.subtype === "success") {
    console.log(message.result);
  }
}
```

Python

```
for await (const message of query({ prompt: "...", options })) {
  if (message.type === "system" && message.subtype === "init") {
    console.log("Available MCP tools:", message.mcp_servers);
  }
}
```

```
const _ = {
  options: {
    mcpServers: {
      github: {
        command: "npx",
        args: ["-y", "@modelcontextprotocol/server-github"],
        env: {
          GITHUB_TOKEN: process.env.GITHUB_TOKEN
        }
      }
    },
    allowedTools: ["mcp__github__list_issues", "mcp__github__search_issues"]
  }
};
```

```
const _ = {
  options: {
    mcpServers: {
```

```
      "remote-api": {
        type: "sse",
        url: "https://api.example.com/mcp/sse",
        headers: {
          Authorization: `Bearer ${process.env.API_TOKEN}`
        }
      }
    },
    allowedTools: ["mcp__remote-api__*"]
  }
};
```

```
const _ = {
  options: {
    mcpServers: {
      github: {
        command: "npx",
        args: ["-y", "@modelcontextprotocol/server-github"],
        env: {
          GITHUB_TOKEN: process.env.GITHUB_TOKEN
        }
      }
    },
    allowedTools: ["mcp__github__list_issues"]
  }
};
```

```
const _ = {
  options: {
    mcpServers: {
      "secure-api": {
        type: "http",
        url: "https://api.example.com/mcp",
        headers: {
          Authorization: `Bearer ${process.env.API_TOKEN}`
        }
      }
    },
    allowedTools: ["mcp__secure-api__*"]
  }
};
```

```
const _ = {
  options: {
    mcpServers: {
      github: {
        command: "npx",
        args: ["-y", "@modelcontextprotocol/server-github"],
        env: {
          GITHUB_TOKEN: process.env.GITHUB_TOKEN
        }
      }
    },
    allowedTools: ["mcp__github__list_issues", "mcp__github__search_issues"]
  }
};
```

```
const _ = {
  options: {
    mcpServers: {
      github: {
        command: "npx",
        args: ["-y", "@modelcontextprotocol/server-github"],
        env: {
          GITHUB_TOKEN: process.env.GITHUB_TOKEN
        }
      }
    },
    allowedTools: ["mcp__github__list_issues", "mcp__github__search_issues"]
  }
};
```

```
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {
        "GITHUB_TOKEN": "${GITHUB_TOKEN}"
      }
    }
  }
}
```

```
  const _ = {
    options: {
      mcpServers: {
        github: {
          command: "npx",
          args: ["-y", "@modelcontextprotocol/server-github"],
          env: {
            GITHUB_TOKEN: process.env.GITHUB_TOKEN
          }
        }
      },
      allowedTools: ["mcp__github__list_issues", "mcp__github__search_issues"]
    }
  };
```

```
  const _ = {
    options: {
```

```
    mcpServers: {
      "remote-api": {
        type: "sse",
        url: "https://api.example.com/mcp/sse",
        headers: {
          Authorization: `Bearer ${process.env.API_TOKEN}`
        }
      }
    },
    allowedTools: ["mcp__remote-api__*"]
  }
};
```

```
const _ = {
  options: {
    mcpServers: {
      github: {
        command: "npx",
        args: ["-y", "@modelcontextprotocol/server-github"],
        env: {
          GITHUB_TOKEN: process.env.GITHUB_TOKEN
        }
      }
    },
    allowedTools: ["mcp__github__list_issues"]
  }
};
```

```
const _ = {
  options: {
    mcpServers: {
      "secure-api": {
        type: "http",
        url: "https://api.example.com/mcp",
        headers: {
          Authorization: `Bearer ${process.env.API_TOKEN}`
        }
      }
    },
    allowedTools: ["mcp__secure-api__*"]
  }
};
```

```
const _ = {
  options: {
    mcpServers: {
      "remote-api": {
        type: "sse",
        url: "https://api.example.com/mcp/sse",
        headers: {
          Authorization: `Bearer ${process.env.API_TOKEN}`
        }
      }
    },
    allowedTools: ["mcp__remote-api__*"]
  }
};
```

```
const _ = {
  options: {
    mcpServers: {
      "remote-api": {
        type: "sse",
        url: "https://api.example.com/mcp/sse",
        headers: {
          Authorization: `Bearer ${process.env.API_TOKEN}`
        }
      }
    },
    allowedTools: ["mcp__remote-api__*"]
  }
};
```

```
{
  "mcpServers": {
    "remote-api": {
      "type": "sse",
      "url": "https://api.example.com/mcp/sse",
      "headers": {
        "Authorization": "Bearer ${API_TOKEN}"
      }
    }
  }
}
```

```
  const _ = {
    options: {
      mcpServers: {
        github: {
          command: "npx",
          args: ["-y", "@modelcontextprotocol/server-github"],
          env: {
            GITHUB_TOKEN: process.env.GITHUB_TOKEN
          }
        }
      },
      allowedTools: ["mcp__github__list_issues", "mcp__github__search_issues"]
    }
  };
```

```
  const _ = {
    options: {
```

```
    mcpServers: {
      "remote-api": {
        type: "sse",
        url: "https://api.example.com/mcp/sse",
        headers: {
          Authorization: `Bearer ${process.env.API_TOKEN}`
        }
      }
    },
    allowedTools: ["mcp__remote-api__*"]
  }
};
```

```
const _ = {
  options: {
    mcpServers: {
      github: {
        command: "npx",
        args: ["-y", "@modelcontextprotocol/server-github"],
        env: {
          GITHUB_TOKEN: process.env.GITHUB_TOKEN
        }
      }
    },
    allowedTools: ["mcp__github__list_issues"]
  }
};
```

```
const _ = {
  options: {
    mcpServers: {
      "secure-api": {
        type: "http",
        url: "https://api.example.com/mcp",
        headers: {
          Authorization: `Bearer ${process.env.API_TOKEN}`
        }
      }
    },
    allowedTools: ["mcp__secure-api__*"]
  }
};
```

```
const _ = {
  options: {
    mcpServers: {
      github: {
        command: "npx",
        args: ["-y", "@modelcontextprotocol/server-github"],
        env: {
          GITHUB_TOKEN: process.env.GITHUB_TOKEN
        }
      }
    },
    allowedTools: ["mcp__github__list_issues"]
  }
};
```

```
const _ = {
  options: {
    mcpServers: {
      github: {
        command: "npx",
        args: ["-y", "@modelcontextprotocol/server-github"],
        env: {
          GITHUB_TOKEN: process.env.GITHUB_TOKEN
        }
      }
    },
    allowedTools: ["mcp__github__list_issues"]
  }
};
```

```
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {
        "GITHUB_TOKEN": "${GITHUB_TOKEN}"
      }
    }
  }
}
```

The `${GITHUB_TOKEN}` syntax expands environment variables at runtime.

TypeScript | Python

```
const _ = {
  options: {
    mcpServers: {
      "secure-api": {
        type: "http",
        url: "https://api.example.com/mcp",
        headers: {
          Authorization: `Bearer ${process.env.API_TOKEN}`
        }
      }
    },
    allowedTools: ["mcp__secure-api__*"]
  }
};
```

```
{
  "mcpServers": {
    "secure-api": {
      "type": "http",
      "url": "https://api.example.com/mcp",
      "headers": {
        "Authorization": "Bearer ${API_TOKEN}"
      }
    }
  }
}
```

The `${API_TOKEN}` syntax expands environment variables at runtime.

```
// After completing OAuth flow in your app
const accessToken = await getAccessTokenFromOAuthFlow();

const options = {
  mcpServers: {
    "oauth-api": {
      type: "http",
      url: "https://api.example.com/mcp",
      headers: {
        Authorization: `Bearer ${accessToken}`
      }
    }
  },
  allowedTools: ["mcp__oauth-api__*"]
};
```

```typescript
import { query } from "@anthropic-ai/claude-agent-sdk";

for await (const message of query({
  prompt: "List the 3 most recent issues in anthropics/claude-code",
  options: {
    mcpServers: {
      github: {
        command: "npx",
        args: ["-y", "@modelcontextprotocol/server-github"],
        env: {
          GITHUB_TOKEN: process.env.GITHUB_TOKEN
        }
      }
    },
    allowedTools: ["mcp__github__list_issues"]
  }
})) {
  // Verify MCP server connected successfully
  if (message.type === "system" && message.subtype === "init") {
    console.log("MCP servers:", message.mcp_servers);
  }

  // Log when Claude calls an MCP tool
  if (message.type === "assistant") {
    for (const block of message.message.content) {
      if (block.type === "tool_use" && block.name.startsWith("mcp__")) {
        console.log("MCP tool called:", block.name);
      }
    }
  }

  // Print the final result
  if (message.type === "result" && message.subtype === "success") {
    console.log(message.result);
  }
}
```

```
import { query } from "@anthropic-ai/claude-agent-sdk";

// Connection string from environment variable
const connectionString = process.env.DATABASE_URL;

for await (const message of query({
  // Natural language query - Claude writes the SQL
  prompt: "How many users signed up last week? Break it down by day.",
  options: {
    mcpServers: {
      postgres: {
        command: "npx",
        // Pass connection string as argument to the server
        args: ["-y", "@modelcontextprotocol/server-postgres", connectionString]
      }
    },
    // Allow only read queries, not writes
    allowedTools: ["mcp__postgres__query"]
  }
})) {
  if (message.type === "result" && message.subtype === "success") {
    console.log(message.result);
  }
}
```

```
import { query } from "@anthropic-ai/claude-agent-sdk";

for await (const message of query({
  prompt: "Process data",
  options: {
    mcpServers: {
      "data-processor": dataServer
    }
  }
})) {
  if (message.type === "system" && message.subtype === "init") {
    const failedServers = message.mcp_servers.filter((s) => s.status !==
```

```
"connected");

    if (failedServers.length > 0) {
      console.warn("Failed to connect:", failedServers);
    }
  }

  if (message.type === "result" && message.subtype === "error_during_execution") {
    console.error("Execution failed");
  }
}
```

OAuth2 authentication

The **MCP specification supports OAuth 2.1** for authorization. The SDK doesn't handle OAuth flows automatically, but you can pass access tokens via headers after completing the OAuth flow in your application:

Examples

List issues from a repository

This example connects to the **GitHub MCP server** to list recent issues. The example includes debug logging to verify the MCP connection and tool calls.

Before running, create a **GitHub personal access token** with `repo` scope and set it as an environment variable:

```
export GITHUB_TOKEN=ghp_xxxxxxxxxxxxxxxxxxxx
```

Query a database

This example uses the **Postgres MCP server** to query a database. The connection string is passed as

an argument to the server. The agent automatically discovers the database schema, writes the SQL query, and returns the results:

Error handling

MCP servers can fail to connect for various reasons: the server process might not be installed, credentials might be invalid, or a remote server might be unreachable.

The SDK emits a `system` message with subtype `init` at the start of each query. This message includes the connection status for each MCP server. Check the `status` field to detect connection failures before the agent starts working:

Troubleshooting

Server shows "failed" status

Check the `init` message to see which servers failed to connect:

```
if (message.type === "system" && message.subtype === "init") {
  for (const server of message.mcp_servers) {
    if (server.status === "failed") {
      console.error(`Server ${server.name} failed to connect`);
    }
  }
}
```

Common causes:

- **Missing environment variables**: Ensure required tokens and credentials are set. For stdio servers, check the `env` field matches what the server expects.
- **Server not installed**: For `npx` commands, verify the package exists and Node.js is in your PATH.
- **Invalid connection string**: For database servers, verify the connection string format and that the database is accessible.
- **Network issues**: For remote HTTP/SSE servers, check the URL is reachable and any firewalls allow the connection.

Tools not being called

If Claude sees tools but doesn't use them, check that you've granted permission with `allowedTools`:

```
const _ = {
  options: {
    mcpServers: {
      // your servers
    },
    allowedTools: ["mcp__servername__*"] // Auto-approve calls
from this server
  }
};
```

Connection timeouts

The MCP SDK has a default timeout of 60 seconds for server connections. If your server takes longer to start, the connection will fail. For servers that need more startup time, consider:

- Using a lighter-weight server if available
- Pre-warming the server before starting your agent
- Checking server logs for slow initialization causes

Related resources

- **Custom tools guide**: Build your own MCP server that runs in-process with your SDK application
- **Permissions**: Control which MCP tools your agent can use with `allowedTools` and `disallowedTools`
- **TypeScript SDK reference**: Full API reference including MCP configuration options
- **Python SDK reference**: Full API reference including MCP configuration options
- **MCP server directory**: Browse available MCP servers for databases, APIs, and more